

MASTER DEVELOPMENT BLUEPRINT

PART-4: DEPLOYMENT, DEVOPS, MONITORING & OPERATIONS

School ERP

Docker | Git | Maven

Prepared by: Solution Architecture / Engineering Leadership Team

Document Version: 1.0

Date: 29 July 2026

Classification: Confidential — Engineering Team Only

Preamble

This document is Part-4 of the Master Development Blueprint (MDB) v1.0 for the School ERP System, continuing directly from Parts 1-3. It defines the mandatory deployment, DevOps, monitoring, and production operations standards, now fixing Docker as the containerization mechanism, Git as version control, and Maven as the backend build tool, applied to the logical deployment architecture already established in Appendix-L and Appendix-N and the test-strategy gating already established in Appendix-M.

This document does not regenerate RGD v2.0, any prior Appendix, any UI/UX Screen Specification, or MDB Parts 1-3. It contains no source code, no Dockerfile, no YAML, no Kubernetes manifests, no shell commands, no CI/CD scripts, no environment variable examples, no configuration files, and no cloud-provider-specific implementation — every standard is stated as a rule or convention only. Where the given Technology Stack does not fix a specific product (e.g., container orchestration platform, CI/CD tool, monitoring product), this document states "Client/Product Decision Required" rather than assuming one.

31. Environment Strategy

Reuses the environment model already fixed in Appendix-L Section 4 and Appendix-N Section 5 (Development, Testing, UAT, Training, Staging, Production) without modification — this section fixes only how Docker realizes that model.

- Every environment runs the Application Layer (Spring Boot 3) and Frontend (React + TypeScript) as Docker containers built from the same image lineage — Development and Production run the same container image where possible, differing only in externalized configuration (Section 32), not in a separately-built artifact.
- PostgreSQL and Redis run as managed or containerized services per environment, scaled per the sizing already fixed in Appendix-N Section 6.8.1 (Development/Testing minimal, Staging/Production full topology).
- The container orchestration platform (e.g., a Kubernetes-class system, a managed container service, or a simpler single-host Docker Compose-class approach for lower environments) is not fixed by the given Technology Stack and remains Client/Product Decision Required — this MDB assumes only that containers are the deployable unit, not a specific orchestrator.
- Environment promotion (Development → Testing → UAT → Staging → Production, Training in parallel) follows the identical sequence and gating already fixed in Appendix-L Section 4.1/25 and Appendix-N Section 25 — not redefined here.

32. Configuration Management

Fixes externalized, environment-specific configuration as the mandatory pattern, realizing the Configuration entity (Appendix-G/H ENT-SYS-005) and Appendix-N Section 32 principle for infrastructure-level settings distinct from application runtime configuration.

- Application configuration is fully externalized from the container image — no environment-specific value (database host, feature flag, integration endpoint) is baked into the Docker image at build time; the same image is promoted unchanged across environments per Section 31.
- Business-policy configuration values already catalogued as pending in Appendix-C v1.1 Section 3.5 (rates, thresholds, SLAs) are sourced from the Configuration entity at runtime (MDB Part-1 Section 8, administration package), not from container-level configuration — these are two distinct configuration mechanisms serving two distinct purposes (infrastructure vs. business policy).
- Configuration changes follow the same Code Review Checklist discipline already fixed in MDB Part-3 Section 30 where they affect application behavior, and the Change Management process (Section 39) where they affect infrastructure.

33. Secrets Management

Fixes handling standards for all credential and key material used across the stack — database credentials, Redis credentials, JWT signing keys (MDB Part-2 Section 13), and external integration credentials (Appendix-I Section 11).

- No secret is ever committed to Git, embedded in a Docker image layer, or logged in application/audit logs (MDB Part-2 Section 20) — this is an absolute rule, not a best-effort guideline.
- Secrets are injected into containers at runtime through the orchestration platform's secret-management mechanism, not baked into the image or passed as plain build arguments — the specific secret-store product remains Client/Product Decision Required, but the injection-at-runtime principle is fixed regardless of product choice.
- JWT signing keys (MDB Part-2 Section 13) are rotated on a defined schedule, with a grace period supporting both old and new keys during rotation so in-flight tokens are not abruptly invalidated.
- Database and Redis credentials differ per environment (Section 31) and are never shared between Production and any non-Production environment, preventing a lower-environment compromise from exposing Production data.

34. Build & Release Standards

Fixes Maven as the backend build tool and a Node-based build for the React + TypeScript frontend, both producing the Docker images deployed per Section 31.

- Every build is versioned using semantic versioning, with the version tied to a specific Git commit/tag — a Docker image is never rebuilt or overwritten under the same version identifier once published.
- The backend Maven build and frontend build both run their respective linting and Unit Testing level (MDB Part-1 Section 10.3, Appendix-M Section 8) as a required build step — a build that fails linting or unit tests does not produce a publishable artifact.
- Built artifacts (Docker images) are published to an artifact/image registry accessible to every environment's deployment mechanism; the specific registry product remains Client/Product Decision Required.
- A release is defined as a specific, tagged combination of backend image version and frontend image version — the two are never deployed as an untracked, ad hoc pairing.

35. CI/CD Standards

Fixes Git-triggered automation as the mandatory mechanism for the Deployment Lifecycle already defined in Appendix-L Section 19 and the Environment Promotion Process in Appendix-N Section 25 — the specific CI/CD platform remains Client/Product Decision Required.

- Every push to a feature branch triggers, at minimum, the build and Unit Testing level (Section 34); every merge to the main integration branch additionally triggers Integration Testing (Appendix-M Section 8) before the resulting image is eligible for promotion to the Testing environment.
- Promotion from Testing → UAT → Staging → Production is gated by the Exit Criteria and Quality Gates already fixed in Appendix-M Sections 21 and 25 — a CI/CD pipeline is not permitted to bypass a gate automatically; Staging → Production specifically requires the human Project Sponsor approval already fixed in Appendix-L Section 20.1/Appendix-M Section 25.
- Pipeline definitions are version-controlled in the same Git repository as the code they build (MDB Part-1 Section 5), so pipeline changes go through the same Code Review Checklist (MDB Part-3 Section 30) as application changes.
- A failed pipeline stage halts promotion at that stage — there is no manual override path that skips a failed automated gate without an explicit, logged exception approved by the Project Sponsor, consistent with the Conditional Pass concept already defined in Appendix-M Section 25.

36. Deployment Strategy

Fixes container-based deployment as the mechanism realizing the Release Deployment Flow already defined in Appendix-N Section 24, without fixing a specific release technique beyond what Appendix-L Section 20 already left open.

- Both the Spring Boot 3 backend and the React + TypeScript frontend are deployed as independently-versioned, independently-scalable Docker containers, consistent with the stateless Application/Web Layer design already fixed in Appendix-I Section 18.1.
- The specific release technique (blue-green, canary, or rolling update) remains Client/Product Decision Required, per Appendix-L Section 20 — whichever is chosen must support the Rollback Strategy already fixed in Appendix-L Section 21 without requiring a schema-level rollback for application-only changes.
- Database schema migrations (MDB Part-3 Section 28) are applied as a distinct deployment step preceding application-container rollout, never bundled into the container startup process itself, so a migration failure is detected and handled before user traffic reaches the new application version.
- Production deployments occur during a planned maintenance window (Appendix-L Section 14, Appendix-N Section 13) wherever possible, consistent with the deliberately-paced release cadence already established in Appendix-L Section 19.1.

37. Monitoring & Health Checks

Fixes Spring Boot's built-in health/metrics exposure as the mandatory mechanism feeding the Monitoring & Alerting architecture already defined in Appendix-L Sections 17, 21-22 and Appendix-N Section 21-22 — the specific monitoring/observability product remains Client/Product Decision Required.

- Every backend container exposes a standard health-check endpoint reporting its own status and the reachability of its immediate dependencies (PostgreSQL, Redis) — the orchestration platform (Section 31) uses this endpoint to determine whether a container receives traffic, directly realizing the Load Balancer health-check mechanism already fixed in Appendix-L Section 6.3/13.1.
- Application-level metrics (response time, error rate, request throughput) are exposed in a standard, scrapeable format matching the Key Metrics already catalogued in Appendix-L Section 17.1, not a bespoke or ad hoc metrics format per module.
- Per-integration health monitoring (Payment Gateway, SMS/Email Gateway, GPS) remains independent per channel, exactly as already fixed in Appendix-F NFR-MON-002 and Appendix-I Section 11.2 — a single combined "integrations healthy/unhealthy" signal is not sufficient.
- Alert thresholds and notification channels for Infrastructure/Operations (Section 39) follow the same escalation path already fixed in Appendix-L Section 22.1 and Appendix-N Section 26.1 — not redefined here.

38. Backup & Disaster Recovery

Fixes PostgreSQL- and Redis-specific realization of the Backup and Disaster Recovery strategy already fully defined in Appendix-L Sections 12-13 and Appendix-N Sections 15-16 — targets and priorities are not redefined here.

- PostgreSQL backup uses continuous write-ahead-log archiving plus periodic full-instance snapshots, supporting point-in-time recovery consistent with the RPO target already fixed in Appendix-L Section 4/Appendix-N Section 4 (exact value Client/Product Decision Required).
- The PostgreSQL read replica (MDB Part-3 Section 21) is the primary mechanism for the failover flow already defined in Appendix-L Section 13/Appendix-N Section 15 — replica promotion is rehearsed at the same DR-drill cadence already fixed in Appendix-F NFR-DR-002.
- Redis, holding refresh-token state (MDB Part-2 Section 13) and cache data (MDB Part-3 Section 26), is configured with persistence sufficient to survive a restart without forcing every active session to re-authenticate, while cache data itself is treated as reconstructable and not backed up with the same rigor as PostgreSQL.
- Container images themselves (Section 34) are inherently recoverable via redeployment from the artifact registry and do not require a separate backup mechanism beyond registry retention.

39. Production Operations Standards

Extends the Operational Responsibilities already fixed in Appendix-L Section 22 and Appendix-N Section 26 with Docker/container-specific operational practice.

- Container restarts, scaling, and rollbacks are executed only through the CI/CD pipeline or orchestration platform (Section 31/35) — direct, manual container manipulation in Production is not permitted outside a declared incident (below).
- An incident (Production outage, Critical-severity defect per Appendix-M Section 14 discovered post-release) triggers the escalation path already fixed in Appendix-L Section 22.1/Appendix-N Section 26.1, with the Rollback Strategy (Appendix-L Section 21) as the primary remediation tool for a bad release.
- Change management for Production — including configuration changes (Section 32), secret rotation (Section 33), and infrastructure scaling — follows the same Separation of Duties principle already fixed in Appendix-L Section 22.2: no single individual holds both change-approval authority and unrestricted Production execution access.
- Every Production change, whether a release (Section 36) or an operational action, is logged with actor, timestamp, and justification, extending the AuditLogService discipline (MDB Part-2 Section 20) to infrastructure-level actions, not only application-level ones.

40. Production Readiness Checklist

Verified before the first Production release and before every subsequent major release, extending the Deployment Validation Checklist already fixed in Appendix-N Section 27 and the Acceptance Criteria already fixed in Appendix-M Section 22.

#	Checklist Item
1	All applicable Functional Requirements (Appendix-E) and Business Rules (Appendix-C v1.1) verified per Appendix-M's Acceptance Criteria (Section 22)
2	Zero open Critical or High severity defects (Appendix-M Section 14) at release time
3	All Non-Functional Requirement targets (Appendix-F) met or formally waived by the Project Sponsor (Appendix-M Section 21)
4	Database migrations (MDB Part-3 Section 28) tested in Staging against production-representative data volume (Appendix-N Section 4)
5	Secrets (Section 33) rotated from any Staging/lower-environment value; no shared credentials between Production and non-Production
6	Health checks (Section 37) verified for every container, including dependency reachability (PostgreSQL, Redis, each external integration)
7	Backup (Section 38) verified via a successful restore test, not merely a completed backup job
8	Disaster Recovery drill (Appendix-N Section 15.1/Appendix-L Section 13.1) successfully rehearsed in Staging at least once
9	Rollback path (Section 36, Appendix-L Section 21) validated for the specific release, not assumed generically viable
10	Monitoring and alerting (Section 37) confirmed operational and routed to the correct on-call/escalation path (Section 39) before go-live
11	Client (Project Sponsor) sign-off obtained per RGD v2.0's Approval Matrix, consistent with Appendix-M Section 22's Acceptance Criteria